Tutorial 1: Introduction to Recursion

Learning Objectives

By completing this tutorial, students should be able to:

- Understand how recursive functions execute
- Identify base cases and recursive cases
- Trace recursive function calls
- Implement simple recursive programs
- Apply recursion to basic problems

Part A: Conceptual Questions

1. Define recursion in your own words.

2. Explain the difference between:

   a) Base case
   b) Recursive case
   Why is the base case important?

3. Consider the function:

```
def countdown(n):
    if n == 0:
        print('Done')
    else:
        print(n)
        countdown(n-1)
```

a) What is the base case?
b) What is the recursive case?
c) What happens if the base case is removed?

4. List two advantages and two disadvantages of recursion.

Part B: Tracing Recursive Execution

5. Trace the execution of:
countdown (3)
Show the output produced.

6. Consider the factorial function:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

Trace the recursive calls for factorial (4).

Call Trace Table:

Call 1: factorial(4)
Call 2: factorial(3)
Call 3: factorial(2)
Call 4: factorial(1)
Call 5: factorial(0)

7. Draw the call stack for factorial(3).

Part C: Writing Recursive Functions

8. Write a recursive function that prints numbers from n down to 0.

9. Write a recursive function that computes the sum of the first n integers.

10. Write a recursive function to compute factorial(n). Test it for n = 5.

Part D: Recursion and Data Structures

11. Explain why recursion is useful when working with tree data structures.

Part E: Challenge Questions

12. Write a recursive function to compute the nth Fibonacci number.

13. Explain why the recursive Fibonacci algorithm is inefficient.

14. Convert the recursive factorial function into an iterative version.